

GROUP MEMBERS

Hardik Agrawal - 2022A7PS0045H (f20220045@hyderabad.bits-pilani.ac.in)

Kalash Bhattad - 2022A7PS0065H (f20220065@hyderabad.bits-pilani.ac.in)

RVS Aashrey Kumar - 2022A7PS0160H (f20220160@hyderabad.bits-pilani.ac.in)

Pratyush Bindal - 2022A7PS0119H (f20220119@hyderabad.bits-pilani.ac.in)

README FOR SYTNAX ANALYSIS

Project Structure

- `parsersyntax.l` – Lex (Flex) specification file
- `parsersyntax.y` – Yacc (Bison) grammar file
- `Makefile` – Automates the build process
- Output: `a.out` (the default compiled parser)

Compilation Instructions

Using Makefile

1. Ensure the `Makefile`, `parsersyntax.l`, and `parsersyntax.y` are in the same directory.
2. In terminal, navigate to the project directory.
3. Run: **make**

This will generate:

- `y.tab.c` and `y.tab.h` from Bison
 - `lex.yy.c` from Flex
 - `a.out` as the final executable
4. To run the parser: **`./a.out input.txt`**
 5. Additionally, to visualise the AST, copy the output on the screen from “Abstract Syntax Tree:” into the python script as in labsheet and run: **`python3 tree.py`**

Manual Compilation (Without Makefile)

1. Generate Bison output: **bison -d parsersyntax.y**

This creates parsersyntax.tab.c and parsersyntax.tab.h.

2. Generate Flex output: **flex parsersyntax.l**

This creates lex.yy.c.

3. Compile using gcc: **gcc -g parsersyntax.tab.c lex.yy.c parserast.c -lfl**
4. Run the parser: **./a.out input.txt**
5. Additionally, to visualise the AST, copy the output on the screen from into the python script as in labsheet and run: **python3 tree.py**